When Can Off-the-Shelf LLMs Classify the GPU Roofline?

Anonymous Author(s)
Anonymous Institution(s)

***Abstract*—GPU kernel optimization hinges on an early decision: is a kernel limited by memory bandwidth or by compute? Answering it today requires profiling on the target GPU, hardware that is often unavailable early in development, in continuous integration, or to the AI coding agents that increasingly propose GPU code changes. We ask whether off-the-shelf large language models can make this call from static code alone, before any profiling run.**

We prompt the models to estimate a kernel's floating-point work and memory traffic—from which its Roofline arithmetic intensity and bandwidth- versus compute-bound (BB/CB) class follow—and evaluate 254 CUDA and OpenMP kernels across four NVIDIA GPUs. Our central finding is about evidence rather than raw accuracy: from source code, the strongest model classifies the FP32 regime correctly for roughly nine in ten kernels, but is no better than a coin flip on FP16; supplying the compiled assembly closes that gap, taking FP16 from chance to near-perfect. The signal is also cheap: a budget model triages FP32 for a fraction of a cent per query and needs no target GPU. We further characterize how stable these calls are under repeated queries. We release a dataset of kernels, build context, assembly, and profiler ground truth, and bound the claim carefully: these models triage the Roofline regime within identifiable evidence tiers, but do not replace profiling.

***Index Terms*—software performance, GPUs, Roofline, arithmetic intensity, large language models, static performance reasoning**

I. Introduction

GPU performance engineering is fundamentally profiler-driven. Developers optimize kernels by compiling candidate variants, running them on the target accelerator, reading hardware counters, and only then deciding whether computation, memory traffic, or both deserve attention—typically through Roofline-style analysis [1]. This workflow is already costly for humans, and it becomes far more expensive in the agentic software-engineering loops that are now reshaping how GPU code is written: with the growing adoption of AI coding *teammates* [2], an LLM increasingly proposes many candidate edits and expects profiler feedback after each one [3]–[5]. Figure 1 sketches this profiler-in-the-loop workflow and the earlier, hardware-free triage step we study.

In practice, target-GPU access is often the real bottleneck. Accelerators may be rationed on a shared cluster, unavailable in early development or continuous integration, blocked by missing system dependencies, or simply too

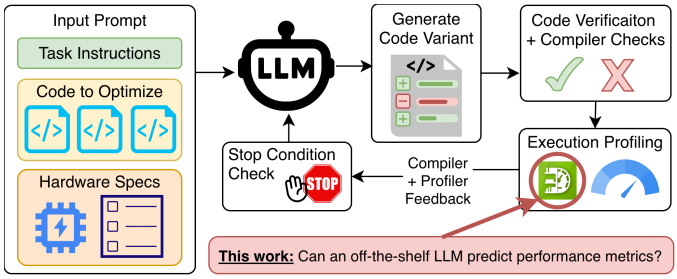

Fig. 1. Profiler-in-the-loop GPU optimization is effective, but it assumes repeated access to the target accelerator. We study whether off-the-shelf LLMs can classify a kernel's bandwidth-bound versus compute-bound Roofline regime earlier in that loop, from software artifacts alone, before runtime profiling is available.

expensive to occupy for speculative triage. The practical question is therefore not whether an LLM can *replace* a profiler, but a narrower one that a developer or coding agent faces *before* any profiling run is available: on a given target GPU, is this kernel likely to be limited by data movement or by computation?

That bandwidth-bound versus compute-bound (BB/CB) decision is the first branch of Roofline analysis [1], and it is the one that decides which optimization direction is even worth attempting. Crucially, it is a *classification*, not a precise number: a developer who learns that a kernel is firmly compute-bound can act on that without knowing its arithmetic intensity to two decimal places. We therefore make Roofline-regime classification the primary object of study, and treat the underlying Roofline Arithmetic Intensity (RAI)—the ratio of floating-point work to bytes moved—as a diagnostic that explains *why* a classification is right or wrong rather than as the headline deliverable. This reframes prior work that asked LLMs only for coarse memory-/compute-bound labels [6] or for raw metric values, and lets us report both an actionable verdict and the numeric reasoning behind it.

The central difficulty is that effective RAI is not always visible in source code. Compiler passes—lowering, instruction selection, transcendental expansion, and other low-level transformations—can change the realized compute and memory traffic in ways the source does not reveal [7]. Classifying the Roofline regime from source alone is thus a genuine test of whether an LLM can recover profiler-relevant behavior from program structure. We therefore study two evidence settings. The *source-only* setting provides only pre-compilation artifacts—source files, build

commands, runtime arguments, and GPU specifications—and corresponds to early development, code review, or agent planning. The *source+SASS* setting adds the target kernel's SASS (GPU assembly) disassembly, exposing the compiler's effect to the model. Critically, SASS is obtained by *cross-compilation* (for example, `nvcc -arch=sm_90` followed by `cuobjdump -sass`) and so requires no target hardware; both settings remain execution-free at prediction time, and comparing them isolates exactly what compiler lowering adds over source-visible semantics.

Unlike prior work that fine-tunes models for GPU metric prediction [8] or studies broader GPU performance modeling [9], [10], our focus is deliberately narrow: BB/CB triage in the zero-training regime, using off-the-shelf LLMs through prompting alone. We evaluate three such models on 254 real-world CUDA and OpenMP kernels from the HeCBench suite [11] across four NVIDIA GPUs—an RTX 3080, A10, A100, and H100. For each kernel the models predict launch dimensions, per-precision FP16/FP32/FP64 floating-point work, and DRAM bytes read and written, from which we derive both RAI and its BB/CB side relative to each GPU's Roofline balance point. We highlight three findings up front: **(1)** from source artifacts alone, the strongest model already classifies the FP32 regime correctly for roughly nine in ten kernels, yet every model—frontier or budget—sits at chance for FP16; **(2)** this FP16 gap is an *evidence* gap rather than a model-capability gap: the decisive FP16 arithmetic is compiler-synthesized and absent from source, and adding cross-compiled SASS closes the gap, lifting FP16 classification from chance to near-perfect for the stronger models; **(3)** the signal is cheap and tiered (a budget model triages FP32 for a fraction of a cent per query with no target hardware) and stable enough under repeated queries to act on, while numeric RAI stays accurate in identifiable regimes, best read as a diagnostic.

Research Questions. We organize the study around four guiding questions:

(RQ1) Static classification: Can off-the-shelf LLMs classify a kernel's BB/CB Roofline regime from source artifacts alone, and for which precisions?
(RQ2) Compilation evidence: What does cross-compiled SASS add to that classification, and why?
(RQ3) Consistency: How consistent are the predictions across repeated queries to the same model?
(RQ4) Cost and baselines: How do these signals compare—in accuracy and per-query cost—against lightweight static baselines, and how does that cost tier across models?

Contributions. This paper contributes the following:

- We formulate execution-free Roofline-regime classification as a software-engineering task for early GPU-kernel triage, with software artifacts as inputs and profiler-derived BB/CB labels as ground truth.
- We show that the source-versus-compilation evidence boundary, not model choice, governs FP16 triage, giving a mechanistic explanation: the decisive FP16 arithmetic is compiler-synthesized and recoverable from cross-compiled SASS without target hardware.
- We characterize the run-to-run consistency of these predictions under repeated queries, identifying where a single query is trustworthy and where a simple majority vote is warranted.
- We quantify the accuracy and per-query cost of off-the-shelf prompting against context-only, deterministic, and lightweight learned static baselines, yielding a deployable cost-tiered cascade among execution-free strategies rather than an LLM-only oracle.
- We release a dataset for hardware-free Roofline reasoning—GPU kernels, compilation and execution context, post-compilation SASS, profiler-derived ground truth, and model outputs—documenting limits that future work can address with repeated-query and reasoning-versus-memorization studies.[1]

The rest of this paper is organized as follows. section II positions our work relative to Roofline analysis, execution-free GPU performance prediction, and LLM-based optimization. section III defines the prediction task, study design, and metrics. section IV answers the research questions, and section V and section VI translate the findings into implications and limitations for ICSE readers. section VII concludes.

II. Related Work

Our work lies at the intersection of Roofline-guided GPU performance modeling, execution-free performance prediction, and LLM-based code reasoning. The closest prior work predicts performance without full profiler access [6], [8], [10], while adjacent areas use LLMs to generate, iteratively optimize, and reason about accelerator code. The distinction that organizes this section is our objective: we treat the profiler-derived BB/CB regime as the primary prediction target and the underlying Roofline Arithmetic Intensity (RAI) as a diagnostic, and we ask what an *off-the-shelf* model can recover from ordinary software artifacts.

Roofline modeling and static GPU performance prediction. The Roofline model is a standard framework for reasoning about whether a kernel is limited by computation or by data movement, with arithmetic intensity as one of its central quantities [1]. Long before LLMs, a substantial body of work predicted aspects of GPU performance (runtime, power, and energy) without executing the target kernel, using analytical and learned models built from program structure, microarchitectural assumptions, or low-level code features. Early analytical models estimated execution time from memory- and thread-level parallelism or from compiler-derived workflow graphs that capture divergence, coalescing, and bank conflicts [12],

[1] https://github.com/FLOPBench/SC26FLOPBench

[13], while later static approaches predicted execution time directly from PTX or other compile-time features [14]–[16]. Subsequent work extended this to GPUs with cache-aware performance, power, and energy models [17], predictors combining PTX features with random forests or XGBoost or with static estimation of execution statistics [18]–[20], bulk-synchronous and DVFS-aware analytical models [21]–[23], and recent learned predictors that transfer across workloads and devices via few-shot learning, GNN/LLM hybrids, or deep forecasters [9], [24]–[26].

These works establish the value of execution-free prediction, but they predict a different quantity (runtime, throughput, power, or energy) rather than the per-precision BB/CB regime we study, so rather than re-implement methods built for another target, we compare against purpose-built lightweight static baselines defined for our exact task (section III). The closest static Roofline work, PolyUFC [27], does characterize CB versus BB regimes without execution, but only for CPU uncore-frequency capping on affine, polyhedral programs, and GPU static analyzers such as FlipFlop [28] target energy and launch-configuration tuning rather than arithmetic intensity. To our knowledge, no prior non-LLM method classifies an arbitrary GPU kernel as BB versus CB, or predicts its arithmetic intensity, from static source or SASS alone.

LLMs for HPC and parallel code understanding and generation. A broad line of work studies whether LLMs can better represent and generate HPC and parallel code through domain specialization, tailored tokenization, and benchmark-driven evaluation. Early domain-specific efforts include *Scope Is All You Need* [29] and *MonoCoder* [30], while HPC-Coder [31] and related work [32] show that HPC-focused corpora and fine-tuning improve performance on parallel-code tasks. Other work studies compiler-oriented representations or benchmark suites for parallel and accelerator code, including Meta LLM Compiler [33], ParEval [34], KernelBench [4], and MultiKernelBench [35], and surveys argue that HPC is a distinct setting for LLMs because correctness, hardware sensitivity, and performance portability matter simultaneously [3], [36]. Relative to this literature, our goal is not to generate kernels but to infer an interpretable, actionable performance regime from existing kernels before optimization begins.

LLMs for iterative GPU code optimization. A large neighboring literature uses LLMs to improve code performance through iterative search with compilation, execution, and profiler feedback in the loop. This includes evaluation-oriented studies such as KernelBench [4], `robust-kbench` [37], and broader assessments of LLMs on HPC optimization [38], [39], as well as feedback-driven systems such as CUDA-LLM [5], PerfCodeGen [40], SpeedGen [41], WarpDrive [42], IntelliPerf [43], Autocomp [44], SwizzlePerf [45], CudaForge [46], ParaCodex [47], SysLLMatic [48], the minimal-executable-program approach of Chu *et al.* [49], and the agent system of Wei *et al.* [50]. A related training-based line uses RL or fine-tuning to optimize GPU code, including Kevin [51], CUDA-L1 [52], KernelBlaster [53], and work integrating performance tools into model reasoning [54], and evolutionary search formulations such as LLM-GP [55] and AlphaEvolve [56] reinforce the value of iterative feedback. These systems show that LLMs can optimize code *when* hardware feedback is available; we instead ask what triage guidance is obtainable *before* it exists, so that such loops could consult an execution-free regime classifier when the accelerator is unavailable.

LLMs for GPU performance prediction. A smaller but much closer body of work uses LLMs as performance predictors rather than optimizers, and the nearest results are all *fine-tuned*. LLMPerf [10] fine-tunes models as GPU performance estimators for source programs, and Omniwise [8] introduces a fine-tuning pipeline that predicts several GPU metrics—including bandwidth, cache behavior, GFLOPs, and arithmetic intensity—directly from code, reporting over 90% of predictions within 10%. Omniwise is closest to our work, but it answers a different question (what a *trained* model predicts) and targets AMD GPUs with no released model retargetable to the NVIDIA GPUs and counters we study, so it cannot be run on our corpus; its strong fine-tuned numbers only sharpen the off-the-shelf gap we characterize. Most directly, Bolet *et al.* [6] framed LLM-based prediction as a *coarse* Roofline classification that labels whole kernels as CB or BB. We are the zero-training, classification-first counterpart to this line: we prompt off-the-shelf models and generalize that single coarse label into a per-precision decomposed signal, predicting launch dimensions, FP16/FP32/FP64 work, and DRAM bytes, from which both the BB/CB regime and RAI follow. We further contrast *source-only* reasoning with prompts containing the kernel's SASS to isolate compiler-realized work that is not syntactically obvious in source [7], [57], [58].

Program reasoning and code-property modeling. Classifying a kernel's Roofline regime also relates to whether LLMs can infer latent execution behavior from source. Work on code execution, predictive execution, and runtime-behavior reasoning shows that models can sometimes simulate program behavior but often struggle as reasoning depth and trace complexity grow [59]–[62], and benchmarks for static-analysis-style reasoning and broader code evaluation similarly find current code LLMs brittle on deeper semantic tasks [63], [64]. Related work further argues that text-only code LLMs struggle to model latent properties such as performance, motivating structured or lower-level representations [65]—which is precisely the role our *source+SASS* setting probes. Regime classification fits this landscape because it requires recovering execution-relevant behavior from code in a form that is directly actionable for performance engineering.

Gap addressed by this work. Taken together, prior work shows that static and learned models can predict

aspects of GPU performance without execution, that LLM-based optimization systems benefit strongly from profiler feedback, that fine-tuned LLMs can estimate GPU metrics, and that off-the-shelf LLMs remain imperfect at reasoning about latent program behavior. What remains underexplored is whether off-the-shelf LLMs can serve as *hardware-free BB/CB triage tools*, classifying a kernel's Roofline regime directly from ordinary software artifacts well enough to guide where scarce profiling and tuning effort should be spent. We address this gap by studying per-precision regime classification across GPU architectures, separating source-visible from post-compilation evidence, and characterizing the regimes where off-the-shelf prompting is reliable, merely useful for triage, or unreliable.

III. Methodology

Prediction Task. Our primary target is a kernel's BB/CB Roofline *regime* (i.e.: classification) on a given GPU: whether, on its *first invocation*, the kernel sits on the memory-bandwidth-limited or the compute-limited side of that device's Roofline balance point. The underlying quantity is the DRAM-level Roofline Arithmetic Intensity (RAI), the ratio of floating-point work to DRAM traffic; comparing RAI against the GPU-specific balance point yields the regime label, and we treat the numeric RAI as a diagnostic that explains *why* a classification is right or wrong. At prediction time the LLM receives software artifacts and must infer the quantities from which RAI, and hence the regime, are derived.

We deliberately scope this task to its simplest useful version via two design decisions: (1) We predict RAI at the DRAM level rather than at each level of the memory hierarchy: a single, interpretable byte denominator is a far easier target than reasoning about arithmetic intensity at every cache level. (2) We evaluate only a kernel's first invocation, whose cold-start cache behavior is easier to reason about than a later invocation that would force the LLMs to essentially simulate the cache state left by prior iterations. Both choices give the LLMs an easier problem on which to measure baseline competence before we move to steady-state, warm-cache prediction, and end-to-end application performance in future work.

We do not ask the LLMs to emit the regime label or RAI directly. For each profiled first-invocation kernel execution on a GPU, the model predicts seven values: grid size, block size, FP16/FP32/FP64 operation counts, and DRAM bytes read and written. We then derive RAI separately for each precision $p \in \{\texttt{FP16}, \texttt{FP32}, \texttt{FP64}\}$,

$$\texttt{RAI}_p = \frac{\texttt{FLOP}_p}{\texttt{BytesRead} + \texttt{BytesWritten}},$$

and classify each precision relative to that GPU's balance point. This decomposition makes failures interpretable, since each quantity exercises a distinct reasoning step: the launch configuration requires propagating command-line inputs through the host code to recover the first executed launch, so an error there propagates into total work; the FLOP count requires reasoning about thread-level work, loop trip counts, divergence, and compiler effects, so an error there signals a semantic or compiler-lowering mistake; and the DRAM-traffic estimate requires reasoning about what reaches on-board DRAM rather than what appears as source-level loads and stores (including the L2 write-back cache), so an error there signals difficulty with DRAM-visible traffic.

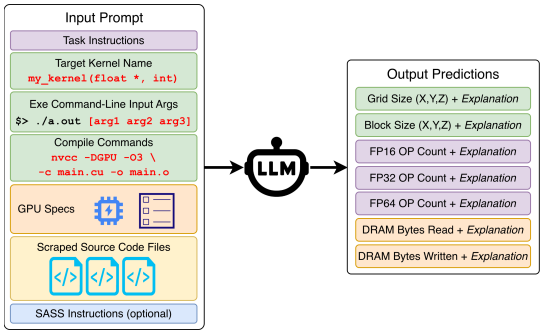

Fig. 2. Per-kernel prompting workflow. Each query includes the target kernel name, source files, compile commands, runtime arguments, and GPU specifications. The *source+SASS* setting adds the target kernel's SASS sections and reachable helper routines so we can measure the value of post-compilation evidence.

We keep separate FP16/FP32/FP64 RAI values rather than collapsing mixed-precision kernels into a single scalar. This matches the per-precision balance points used for triage and makes compiler-injected FP16 work visible, so a kernel may have zero FP64 RAI but nonzero FP32 RAI.

Input Evidence and Prompting. Figure 2 summarizes the evidence supplied to the model for one target kernel on one GPU. For OpenMP kernels we additionally preserve the source location needed to disambiguate the target-offload region, since one executable may contain several kernels but the model reasons about only one. Compile commands matter because macros, include paths, and optimization flags affect both the visible source and the compiled behavior, and source files are embedded directly, wrapped in XML-style tags, so the model reasons over the exact code used to build the executable. The prompt is split into a system message that defines the task and output constraints and a task message that carries the kernel, source, build, runtime, and hardware context, where we enforce structured output through tool calling so each response returns the seven predicted quantities with a short explanation for each; the explanations support sanity checking but are not used to compute any reported metric. A full prompt and response example appears in our GitHub repo [2].

As part of LLM prompting, we study two evidence settings that differ by the presence of post-compilation SASS. ***source-only*** corresponds to early development, code review, or agent planning, when only pre-compilation

[2] https://github.com/FLOPBench/SC26FLOPBench

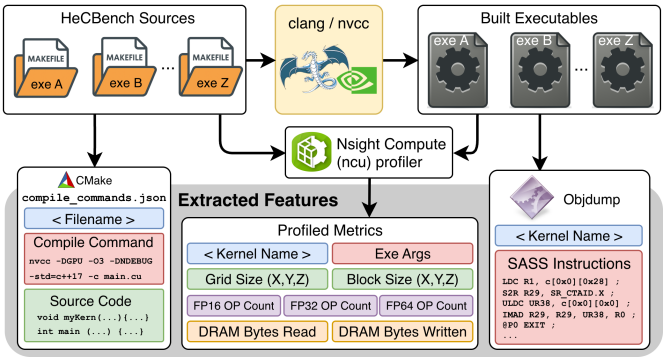

Fig. 3. Dataset-construction pipeline. For each target kernel we build the executable, recover the source files, compile commands, and runtime arguments, cross-compile to obtain SASS, and profile the first invocation to collect the floating-point and DRAM-traffic ground truth used to label the Roofline regime.

TABLE I
PROFILED GPUS AND DRAM-LEVEL BALANCE POINTS USED FOR BB/CB CLASSIFICATION. PEAK THROUGHPUT IS IN TFLOP/S; BALANCE POINTS (BP) ARE IN FLOP/BYTE.

	RTX 3080	A10	A100	H100
Architecture	GA102	GA102	GA100	GH100
Compute cap.	8.6	8.6	8.0	9.0
BW (GB/s)	760	600	1,555	3,360
L2 (MB)	5	6	40	50
FP16 peak	30.55	15.62	77.97	133.82
FP32 peak	30.55	15.62	19.49	66.91
FP64 peak	0.477	0.244	9.75	33.45
FP16 BP	40.20	26.03	50.14	39.83
FP32 BP	40.20	26.03	12.53	19.91
FP64 BP	0.63	0.41	6.27	9.96

artifacts are available. ***source+SASS*** instead adds SASS (i.e. GPU assembly) to the prompt, exposing the compiler's final instruction selection and code generation decisions. The SASS is obtained by cross-compilation, so it requires no target hardware access. Crucially, comparing the *source-only* and *source+SASS* evidence settings isolates what compiler lowering adds over source-visible semantics.

Corpus and Ground Truth. We draw workloads from the HeCBench suite [66], focusing on CUDA and OpenMP target-offload programs because they are both common GPU programming models and represent distinct source/build ecosystems [67]. Our final corpus contains 95 programs (56 CUDA, 39 OpenMP), which expose 254 first-invocation GPU kernels (155 CUDA, 99 OpenMP); we preserve the suite's natural CUDA/OpenMP mix rather than artificially balancing the two. As summarized in Figure 3, ground-truth RAI labels come from an offline profiling process: we profile each program's first kernel invocation, averaging over three runs to reduce noise, and derive per-precision FLOP counts and DRAM byte traffic from the profiler counters. Profiling the corpus on four NVIDIA GPUs yields 1,016 samples, and each sample induces three precision-specific RAI labels (FP16/FP32/FP64), so the profiler-derived ground-truth space contains 3,048 precision-specific evaluation points.

We profile on four NVIDIA GPUs that span different memory and compute regimes: RTX 3080, A10, A100, and H100. Table I lists their characteristics and the resulting DRAM-level balance points used for BB/CB classification. The architectural question is therefore not whether LLMs generalize to all accelerators; it is whether the same prompting strategy remains useful across noticeably different NVIDIA deployment targets.

TABLE II
GROUND-TRUTH RAI CLASS DISTRIBUTION BY GPU AND PRECISION OVER THE FULL CORPUS. SUB-COLUMNS COUNT, AMONG THE 254 FIRST-INVOCATION KERNELS, THOSE WITH ZERO RAI (0 FLOP) AND NONZERO BB/CB LABELS.

	FP16			FP32			FP64		
GPU	0	BB	CB	0	BB	CB	0	BB	CB
RTX 3080	249	5	0	169	70	15	217	10	27
A10	249	5	0	169	68	17	217	8	29
A100	60	169	25	169	65	20	217	31	6
H100	129	103	22	169	66	19	217	31	6

TABLE III
STUDIED OFF-THE-SHELF LLMS, SORTED BY RELEASE DATE.

Model	Released	$/1M (in/out)	Max tok. (in/out)
`GPT OSS 120B`	2025-08-05	0.039 / 0.19	131K / 131K
`Opus 4.6`	2026-02-04	5.00 / 25.00	872K / 128K
`GPT-5.4`	2026-03-05	2.50 / 15.00	922K / 128K

Table II highlights the resulting RAI label class imbalance: across the full 1,016 GPU–kernel profiling samples, FP16 has 687 zero-FLOP cases, FP32 has 676, and FP64 has 868. FP16 is especially architecture-sensitive because the A100/H100 binaries contain many more nonzero FP16 measurements than the 3080/A10. This imbalance is why we separate the evaluation tasks by precision instead of reporting single aggregate errors.

Error-Prone Code Features. We additionally run an auxiliary feature-voting pipeline in which inexpensive LLMs label each kernel's source for patterns that are known to complicate floating-point and traffic reasoning: division, special math (transcendental and intrinsic functions), reusable subexpressions, and loop-invariant floating-point work. We use these labels in two ways: to stratify the repeated-query panel into *harder* and *easier* kernels, and, after evaluation, to characterize where prediction errors cluster. The labels are not human-validated, so we use them to describe recurring failure patterns rather than to make causal claims, and we summarize association strength with Cliff's δ.

Models and Experiments. We study three off-the-shelf LLMs (Table III): the two frontier models `GPT-5.4` and `Opus 4.6`, and the roughly 100× cheaper open-source baseline `GPT OSS`. We do not fine-tune the models, we only vary prompting by the two evidence configurations (*source-only* and *source+SASS*), and supplied GPU specifications from Table I. Because querying every input more than once across all three models is prohibitively expensive, our primary study collects exactly one sample per (kernel, GPU, evidence) combination on each LLM. Over the shared corpus of 254 kernels on four GPUs

(1,016 kernel–GPU samples per evidence setting), the `GPT OSS` model repeatedly failed to return valid responses for a substantial fraction of inputs, while `GPT-5.4` and `Opus 4.6` completed all samples in both evidence settings. `GPT OSS`'s shortfalls primarily reflect context-overflow or completion failures on long prompts. For fair cross-model comparison we restrict our analysis to the *intersection* of samples completed by all three LLMs in both evidence settings, which leaves 732 kernel–GPU samples.

A single sample per input cannot establish reliability, since LLM outputs are nondeterministic, so we additionally run a smaller repeated-query study to characterize run-to-run variability without re-running the full grid. We select a small, deterministic, stratified panel of 24 kernels from the evaluated set, chosen to span precision, proximity to the balance point, BB/CB class, runtime (CUDA/OpenMP), and a hard-versus-easy split based on the error-prone code features above, rather than only low-error cases. We resample each (kernel, GPU, evidence) combination four times on every model, collecting 384 samples per model per evidence setting (96 kernel–GPU samples × four repeats), or 2,304 repeated-query samples in total. From these we measure two kinds of consistency: the coefficient of variation of predicted RAI across repeats, and whether the BB/CB classification is unanimous across repeats.

Evaluation Metrics. We report a deliberately small set of metrics organized around the primary outcome (BB/CB regime classification) and a numeric diagnostic (RAI error).

a) Regime classification (primary).: For nonzero cases we compare the predicted and expected side of the GPU-specific balance point: a kernel is CB when its RAI exceeds that balance point and BB otherwise. Because the class ratios are uneven, we report balanced accuracy (BAcc, macro recall), which cannot be inflated by predicting the majority class, and we additionally report CB precision/recall/F1 so that high recall coming from over-predicting the compute side stays visible. If a model predicts zero for a truly nonzero kernel, we conservatively count it as a memory-side failure, since a zero prediction still fails to identify compute-side behavior.

b) Numeric RAI error (diagnostic).: For nonzero cases we report the median absolute percent error (MdAPE) of predicted RAI, where the percent error for precision p is $100\,(\text{predicted}_p - \text{expected}_p)/\text{expected}_p$. The distributions are heavy-tailed, so we emphasize medians and interquartile behavior rather than means.

c) Zero detection.: Because the corpus is zero-heavy, we also report balanced accuracy on distinguishing zero from nonzero RAI; only an exact zero prediction counts as "zero," and any positive or non-finite prediction is conservatively treated as "nonzero." As reference points, always predicting zero yields 0.50 balanced accuracy on zero detection, and always predicting the majority nonzero class yields 0.50 on regime classification.

Static Baselines. To test whether the LLM signal is worth its cost, we compare each model against lightweight static predictors defined for the same per-precision DRAM-level target and scored on the same 732-sample shared subset, ground truth, and balance points, so the comparison is like-for-like. We use two families. The first is a set of *deterministic* references that compute RAI analytically with no training: a *source-lexical heuristic* that counts arithmetic tokens over memory references in the comment-stripped source and gates each precision on its type keywords; a *SASS-mnemonic heuristic* that forms a per-precision floating-point count from the compiled instruction mix (weighting fused and packed instructions by their operation count) over global-memory instructions times the precision's byte width; and a *context-median predictor* that returns the median RAI of the training kernels within each runtime×GPU group. The second is a set of *learned* tree models—Random Forest and Extra Trees—trained on source-derived features alone (matching *source-only*) or on source plus SASS-derived features (matching *source+SASS*), where features are token and instruction mix counts with runtime and GPU as categoricals. Like the LLMs, these predict per-precision RAI through a two-stage estimator (zero-vs-nonzero, then log-space regression), from which the BB/CB label follows by comparison against the GPU balance point. We evaluate with leave-programs-out (grouped five-fold) cross-validation and average over five seeds for stability. Exact token, mnemonic, and feature definitions are in the released code.

IV. Evaluation

This section evaluates whether off-the-shelf LLMs can act as useful Roofline triage tools for GPU kernels, organized around the four research questions of section I.

Unless stated otherwise, the cross-model comparisons use the 732-sample shared subset of section III. We report nonzero BB/CB regime classification as the primary outcome and treat the numeric RAI error as a diagnostic that explains *why* a classification succeeds or fails. Because the RAI-percent-error distributions are heavy-tailed, we emphasize balanced accuracy, medians, and interquartile behavior rather than means. In the tables that follow, we abbreviate the two evidence settings as SO (*source-only*) and S+S (*source+SASS*).

A. RQ1: Can LLMs Classify the Roofline Regime From Static Artifacts?

For FP32 and FP64, yes; for FP16, not from *source-only* evidence. Table IV reports nonzero BB/CB balanced accuracy by model, precision, and evidence setting. There are a few trends to note: **(1)** From *source-only* prompts, `Opus 4.6` classifies the FP32 regime at 0.940 balanced accuracy and FP64 at 0.755; `GPT-5.4` reaches 0.889/0.670, and even the budget `GPT OSS` reaches 0.875/0.690. These are strong numbers for a pre-compilation signal: the models recover the compute-versus-memory branch for most

TABLE IV

PER-PRECISION PERFORMANCE ON THE 732-SAMPLE SHARED SUBSET. FOR EACH PRECISION WE REPORT MDAPE (MEDIAN ABSOLUTE PERCENT ERROR ON NONZERO RAI), BACC (BALANCED ACCURACY ON NONZERO BB/CB), AND NONZERO CB PRECISION/RECALL/F1; THE LAST TWO COLUMNS GIVE MEDIAN PER-QUERY COST ($) AND WALL-CLOCK TIME (S). A DASH MARKS AN UNDEFINED VALUE (NO PREDICTED CB KERNELS, HENCE A ZERO DENOMINATOR), AS EVERY MODEL SHOWS FOR SO FP16.

		FP16			FP32			FP64				
Model	Evidence	MdAPE	BAcc	P/R/F1	MdAPE	BAcc	P/R/F1	MdAPE	BAcc	P/R/F1	Med. $	s
`GPT-5.4`	SO	100.0	0.500	– / 0.000 / –	59.2	0.889	0.909 / 0.800 / 0.851	74.5	0.670	0.950 / 0.358 / 0.521	0.0197	11.3
`GPT-5.4`	S+S	84.2	0.870	0.730 / 0.794 / 0.761	34.9	0.895	0.952 / 0.800 / 0.870	35.4	0.737	0.903 / 0.528 / 0.667	0.0315	12.3
`GPT OSS`	SO	100.0	0.500	– / 0.000 / –	58.9	0.875	0.950 / 0.760 / 0.844	62.4	0.690	0.917 / 0.415 / 0.571	0.0006	33.2
`GPT OSS`	S+S	100.0	0.583	0.750 / 0.176 / 0.286	61.4	0.865	0.949 / 0.740 / 0.831	58.4	0.652	0.900 / 0.340 / 0.493	0.0008	32.5
`Opus 4.6`	SO	100.0	0.500	– / 0.000 / –	43.9	0.940	1.000 / 0.880 / 0.936	52.0	0.755	0.966 / 0.528 / 0.683	0.0679	22.5
`Opus 4.6`	S+S	20.4	0.984	0.850 / 1.000 / 0.919	18.3	0.967	0.979 / 0.940 / 0.959	15.3	0.765	0.938 / 0.566 / 0.706	0.1066	29.8

FP32 and FP64 kernels using only source code, build context, runtime arguments, and GPU specifications. **(2)** The FP16 case is a clean failure. From *source-only* prompts, every model—including the frontier ones—sits exactly at the 0.500 chance baseline for FP16. The same models that classify FP32 and FP64 regimes well are no better than a coin flip on FP16 under *source-only* evidence; we return to why in RQ2.

Because the panel is CB-poor (section III), we confirm the strong numbers are not an over-prediction artifact: the CB precision/recall breakdown (Table IV) shows *source-only* `Opus 4.6` reaches 1.000 CB precision at 0.880 recall for FP32, and 0.966 precision at 0.528 recall for FP64. The lower FP64 recall is honest—the models miss roughly half of the compute-bound FP64 kernels even when they place the rest correctly.

The takeaway is that **(a)** off-the-shelf LLMs are already reliable FP32 and FP64 regime classifiers from *source-only* artifacts, with a clear "you-get-what-you-paid-for" ordering from `GPT OSS` to `Opus 4.6`, and **(b)** FP16 triage is not available from *source-only* evidence and must escalate to more evidence, which motivates RQ2.

RQ1 Summary: From *source-only* artifacts, off-the-shelf LLMs reliably classify the FP32 and FP64 Roofline regimes—`Opus 4.6` reaches 0.940 balanced accuracy on FP32 and even the budget `GPT OSS` reaches 0.875—but under the same *source-only* evidence every model, frontier ones included, is no better than a coin flip on FP16.

B. RQ2: What Does Cross-Compiled SASS Add, and Why?

It closes the FP16 gap, and the mechanism is compiler-synthesized arithmetic. Adding *source+SASS* lifts FP16 BB/CB balanced accuracy from the 0.500 *source-only* baseline to 0.984 for `Opus 4.6` and 0.870 for `GPT-5.4` (Table IV). The gain is specific to the evidence, not the model: the same frontier models were at chance for FP16 under *source-only*, while `GPT OSS`—which makes weaker use of long SASS prompts—only reaches 0.583. *source+SASS* helps FP32 and FP64 much more modestly (`Opus 4.6` FP32 balanced accuracy rises 0.940→0.967, FP64 0.755→0.765), because FP32 and FP64 arithmetic is already mostly source-visible.

The reason is mechanistic and visible in the collected samples. FP16 throughput on these GPUs comes from packed `HFMA2` instructions that the compiler synthesizes; they are not syntactically present in source, so *source-only* reasoning conservatively predicts zero FP16 work. This is why the result generalizes beyond a single model: *FP16 Roofline reasoning requires compilation, not execution*, and cross-compilation supplies it without access to the target GPU.

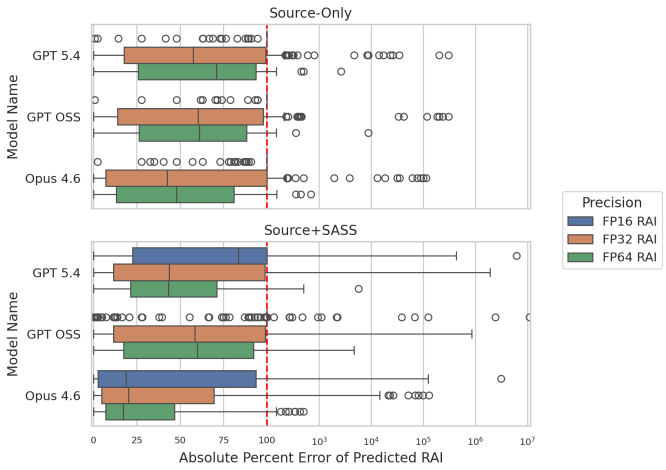

Fig. 4. Absolute percent error on nonzero RAI cases, by model, precision, and evidence setting. The distribution has both a useful low-error region and a long failure tail. *source+SASS* sharply expands the low-error FP16 region for the frontier models and tightens many FP32/FP64 distributions, mirroring the classification gains.

Numeric RAI as a diagnostic. The numeric error tracks the same evidence boundary, which is why we read it as a diagnostic rather than a headline. Figure 4 shows the distribution of nonzero RAI absolute percent error. Where classification is strong the numeric estimate is also close: under *source+SASS*, `Opus 4.6` lands within 25% error of the profiled RAI on 47.0% of nonzero FP16, 53.0% of FP32, and 44.0% of FP64 samples, with median errors of 20.4%, 18.3%, and 15.3%. Where *source-only* is blind (FP16), numeric accuracy collapses to under 1% of samples within 25%. The distributions are heavy-tailed (i.e. some kernels are predicted almost exactly while others are orders of magnitude off) so a deployed tool should treat numeric RAI as a coarse hint and rely on the more robust regime classification.

Error-prone code features. Using the source-feature labels of section III, the residual numeric errors con-

TABLE V

CLIFF'S δ BETWEEN FEATURE-PRESENT AND FEATURE-ABSENT NONZERO RAI ERROR DISTRIBUTIONS. POSITIVE $\Rightarrow$ HIGHER ERROR WHEN THE FEATURE IS PRESENT.

	GPT-5.4		GPT OSS		Opus 4.6	
Feature	SO	S+S	SO	S+S	SO	S+S
Has FLOP Division	0.32	0.34	0.33	0.34	0.34	0.36
Has Float Subexprs	0.30	0.32	0.29	0.32	0.30	0.31
Has Special Math	0.26	0.26	0.25	0.28	0.25	0.29
Has Loop-Invariant FLOPs	0.24	0.27	0.22	0.21	0.23	0.27
Calls Helper Function	0.11	0.14	0.10	0.15	0.15	0.15
Has RNG Inputs	0.11	0.12	0.11	0.13	0.12	0.14
Has Branching	0.08	0.09	0.11	0.11	0.10	0.09
Uses Preproc Defines	0.06	0.08	0.07	0.06	0.08	0.09
Has Data-Dep Branching	0.07	0.07	0.04	0.06	0.07	0.08
Uses Input Data from File	0.02	0.01	0.05	0.04	0.02	0.01
Deterministic Grid Size	0.01	0.01	0.01	0.01	0.03	0.02
Deterministic Block Size	−0.04	−0.03	−0.08	−0.09	−0.07	−0.04

centrate where the same code patterns appear regardless of model or prompt type (Table V). Four FLOP-perturbing features dominate across every model and evidence setting—FLOP division, reusable float subexpressions, transcendental ("special") math functions, and loop-invariant FLOP work, each with a Cliff's δ around 0.2–0.36—whereas structural features such as deterministic launch bounds show essentially no association. These features either inject FLOPs the compiler lowers from a single source token (division, transcendentals) or remove repeated work the model over-counts statically (subexpression elimination, loop-invariant hoisting), so they directly perturb the FLOP term of RAI. The practical implication is that users should expect the largest RAI errors—and should most want SASS—on kernels dense in these features.

RQ2 Summary: Cross-compiled *source+SASS* closes the FP16 gap, lifting FP16 accuracy from 0.500 to 0.984 for `Opus 4.6`, because it exposes the compiler-synthesized packed-half (`HFMA2`) arithmetic that source hides; FP32 and FP64, already source-visible, gain little. FP16 Roofline reasoning therefore requires post-compilation evidence for accurate regime classification.

C. RQ3: How Consistent Are the Predictions Across Repeated Queries?

LLM predictions are nondeterministic, so a single query per kernel cannot establish reliability. We resample the models on the small, stratified panel mentioned in section III and measure two kinds of consistency: the median coefficient of variation (MCV) of the predicted RAI across repeats, and whether the BB/CB prediction is unanimous across repeats (i.e. agreement). Table VI reports both metrics.

Three trends stand out. **(1)** The decision-level prediction is generally stable: it is unanimous across repeats for most kernels, with per-configuration agreement mostly in the 82–100% range, so the headline classification is not an artifact of a single lucky draw. **(2)** The most accurate model is also the most consistent. `Opus 4.6` holds a near-zero MCV and 88–100% agreement in *both* evidence settings, and, unlike the other models, adding SASS barely

TABLE VI

REPEATED-QUERY VARIABILITY. MCV IS THE MEDIAN COEFFICIENT OF VARIATION OF PREDICTED RAI ACROSS REPEATS (LOWER IS STEADIER); AGR IS THE FRACTION OF KERNELS WITH UNANIMOUS BB/CB PREDICTIONS ACROSS REPEATS.

		FP16		FP32		FP64	
Model	Evidence	MCV	Agr	MCV	Agr	MCV	Agr
`Opus 4.6`	SO	0.00	100%	0.18	88%	0.00	100%
`Opus 4.6`	S+S	0.00	96%	0.19	92%	0.08	100%
`GPT-5.4`	SO	0.00	100%	0.21	71%	0.04	82%
`GPT-5.4`	S+S	0.87	86%	0.61	76%	0.33	82%
`GPT OSS`	SO	0.00	100%	0.28	88%	0.03	88%
`GPT OSS`	S+S	0.00	95%	0.53	69%	0.41	69%

perturbs it (FP16 MCV stays 0.00 at 96% agreement). The model one would actually deploy is therefore also the most reliable, and for `Opus 4.6` single-shot use is comparatively safe. **(3)** Where SASS does destabilize predictions, the effect is model-specific rather than intrinsic to the evidence: `GPT-5.4`'s MCV jumps from essentially zero under *source-only* to 0.87 (FP16), 0.61 (FP32), and 0.33 (FP64) once SASS is added, and `GPT OSS`'s *source+SASS* agreement falls to 69% for FP32 and FP64. This is the cost of the evidence that cracks FP16 for the weaker models: once a model commits to a numeric FP16 estimate rather than a degenerate zero, that estimate varies more across repeats.

RQ3 Summary: Decision-level predictions are mostly stable (82–100% agreement across repeats). `Opus 4.6` is both the most accurate and the most consistent, so a single query usually suffices; the cheaper models and `GPT-5.4` under *source+SASS* vary enough that a deployed tool should issue a few repeated queries and take the majority BB/CB vote.

D. RQ4: How Do the Signals Compare in Accuracy and Cost?

The LLM signals form a clear cost/accuracy tier, they are inexpensive in absolute per-query terms, and (unlike profiling) they need no target-hardware access.

Cost tiering. Table IV reports median per-query cost, per LLM-evidence configuration. The budget `GPT OSS` classifies the *source-only* FP32 regime at 0.875 balanced accuracy for a median $0.0006 per query; `GPT-5.4` reaches 0.889 for $0.0197; and `Opus 4.6` reaches 0.940 for $0.0679. For FP16, only the SASS-backed frontier tier works, at $0.0315 (`GPT-5.4`) to $0.1066 (`Opus 4.6`) per query. The *source-only* FP32 tier is inexpensive in absolute terms: at a median $0.0006 per kernel it needs no target-hardware access at all, unlike profiling. We do not present end-to-end build-and-profile timings, so we make no measured time- or cost-saving claim against profiling; the advantages we establish are absolute query cost and the elimination of target-hardware access. The frontier+SASS tier that additionally cracks FP16 is the premium option, and its higher per-query cost is the case where directly profiling may be preferable if the target hardware is available.

Versus static baselines. To establish that the LLM signal is worth its cost beyond off-the-shelf prompting, Table VII compares the best off-the-shelf LLM against

TABLE VII

BEST OFF-THE-SHELF LLM VERSUS BEST STATIC REFERENCE FOR NONZERO BB/CB TRIAGE ON THE SHARED SUBSET. EACH CELL GIVES BACC AND NONZERO MDAPE FOR THE STRONGEST METHOD IN THAT EVIDENCE FAMILY; DISCORDANT-PAIR COLUMNS REPORT EXACT PAIRED WINS ON THE SAME SAMPLES.

Evidence	Precision	Best LLM	BAcc / MdAPE	Best static	BAcc / MdAPE	Discordant wins LLM/static (p)
source-only	FP16	`GPT-5.4`	0.500 / 100.0	Source learned ET	0.723 / 217.7	10/17 (0.248)
source-only	FP32	`Opus 4.6`	0.940 / 43.9	Source learned RF	0.750 / 100.0	52/3 (< 0.001)
source-only	FP64	`Opus 4.6`	0.755 / 52.0	Lexical Heuristic	0.683 / 99.5	21/13 (0.229)
source+SASS	FP16	`Opus 4.6`	0.984 / 20.4	SASS learned ET	0.608 / 163.1	33/5 (< 0.001)
source+SASS	FP32	`Opus 4.6`	0.967 / 18.3	SASS learned ET	0.730 / 100.0	57/4 (< 0.001)
source+SASS	FP64	`Opus 4.6`	0.765 / 15.3	SASS learned RF	0.841 / 62.4	5/13 (0.096)

the best static reference—the learned random-forest (RF) and extra-trees (ET) regressors of section III, alongside lightweight lexical heuristics—for nonzero BB/CB triage, per precision and evidence setting on the 732-kernel subset. Because both methods run on the same kernels, we compare them with a paired (McNemar) test: the discordant-wins column counts kernels where exactly one method is correct, reported as LLM wins/static wins, with p testing whether the LLM wins significantly more of these disagreements. Where the signal matters most the LLMs win decisively and significantly on the same samples: *source-only* FP32 (`Opus 4.6` 0.940 vs. 0.750 balanced accuracy, 52/3 discordant wins), *source+SASS* FP16 (0.984 vs. 0.608, 33/5), and *source+SASS* FP32 (0.967 vs. 0.730, 57/4), and in each of these the LLM's numeric RAI error is far lower (MdAPE 18–44% vs. 100–163%). The static baselines are competitive only in the two regimes where the LLMs are weakest: *source-only* FP16, where neither method is usable—the best LLM is degenerate at 0.500 while the best static reaches 0.723 but with a 218% MdAPE—and *source+SASS* FP64 triage, where a learned tree edges `Opus 4.6` on balanced accuracy (0.841 vs. 0.765) but not significantly ($p = 0.096$) and still with a worse numeric error (62% vs. 15% MdAPE). Across the six regimes the LLMs significantly beat the best static baseline in three—the high-value SASS-backed and FP32 frontier regimes—and the remaining differences are not statistically significant; even where a static baseline edges ahead on the classification call, its RAI estimate is too coarse to be used as a numeric diagnostic.

RQ4 Summary: In the high-value regimes the LLMs significantly beat lightweight static baselines (e.g. *source+SASS* FP16 0.984 vs. 0.608 balanced accuracy, $p < 0.001$), at fractions of a cent per query and with no target-hardware access. No single configuration wins every regime, so the cost-effective choice is to match the model and evidence tier to the target precision.

The takeaway across the four RQs is that off-the-shelf LLMs are a practical, hardware-free Roofline triage tool once matched to the available evidence: *source-only* artifacts suffice for FP32 and FP64 at fractions of a cent, cross-compiled SASS is what unlocks FP16, and single-shot use is safe for a frontier `Opus 4.6` model but should become a small majority vote for the cheaper `GPT-5.4` and `GPT OSS` models.

V. DISCUSSION

What this enables for AI-assisted software engineering. The central positive result—that off-the-shelf LLMs can classify a kernel's BB/CB Roofline regime from software artifacts, without touching the target accelerator—is directly relevant to AI coding agents and developer-facing tools. Before spending target-GPU time, an agent can ask a narrower question than "will this be faster?": is this kernel limited by data movement or by computation, and is the available evidence enough to answer that yet? The answer is evidence-dependent, and numeric RAI serves as a secondary diagnostic: when it is accurate it reinforces the regime call, and when it falls in the heavy-tailed failure region it signals that the sample should be escalated to profiling.

Where *source-only* and *source+SASS* fit in a real workflow. Execution-free reasoning has two meaningful stages. *source-only* fits pre-compilation tasks such as code review, design exploration, and agent planning; *source+SASS* fits a later stage where the code can be compiled for a target architecture but the accelerator is still unavailable for execution or profiling.

Why deterministic and learned static features still matter. The reference baselines sharpen the methodological boundary: a simple lexical rule cannot explain the useful FP32/FP64 *source-only* signal, yet supervised source baselines recover corpus-level triage cues and architecture-specific SASS counts expose substantial post-compilation structure. Our results suggest a cascade combining deterministic compiler or disassembly features, lightweight supervised models, and LLM reasoning could outperform any single strategy by routing each evidence tier to the method that wins it. Table VIII records the per-regime first-stage choice the released evidence supports—a routing guide, not a measured system, since we have not built the combined cascade and choosing a route presumes the precision and evidence tier are known in advance.

What tool builders should and should not take from the results. Tool builders should use the method as a triage filter—to prioritize profiling effort, annotate likely memory- versus compute-side cases, or flag kernels too uncertain to trust—not as a runtime substitute, speedup estimator, or universal static analysis of GPU performance.

Why the dataset release matters. Even with the method's limitations, the released corpus is a useful con-

TABLE VIII
DEPLOYMENT ROUTING SUMMARY FROM THE SHARED SUBSET. THESE ARE CORPUS-BACKED FIRST-STAGE CHOICES, NOT UNIVERSAL RULES.

Regime	First stage	Why
source-only FP16	Static-first	One-shot LLMs stay at the trivial BB/CB baseline; cheap source features recover more queueing signal.
source-only FP32/FP64	Frontier LLM	Best pre-compilation triage overall, especially for FP32 where paired LLM wins are decisive.
source+SASS FP16/FP32	Frontier LLM	Largest paired gains once lowered arithmetic and DRAM-visible behavior become explicit.
source+SASS FP64	Frontier LLM	Learned SASS features edge the label call, but not significantly, and their numeric RAI is several times coarser, so the frontier LLM stays the better single choice.

tribution in its own right: it exposes a well-posed software-engineering task with source files, build context, runtime arguments, SASS, profiler-derived labels, and model outputs, and the added deterministic and learned reference baselines make it a testbed for comparing execution-free reasoning strategies rather than only for ranking LLMs. Triaging the entire 732-sample shared subset with the budget model costs under $0.50, which keeps such comparisons cheap to run. It supports several follow-on directions that require new experiments: stronger compiler-grade analyzers, repeated-query robustness studies, and hybrid predictors that combine compiler features with LLM reasoning.

VI. Threats to Validity and Limitations

External validity. Our claims are scoped to the task and corpus defined in section III: 95 HeCBench programs exposing 254 first-invocation kernels, four NVIDIA GPUs, and per-precision DRAM-level RAI on a kernel's first invocation. Our corpus is a meaningful but selective sample of CUDA and OpenMP software, and the architectural generalization claims extend only to these device families. The dataset does not cover AMD or Intel GPUs, other programming models such as HIP and SYCL, steady-state warm-cache behavior, or end-to-end application performance, and the results should not be extrapolated beyond execution-free first-invocation triage.

Construct validity. Ground truth comes from profiler counters rather than source-level reasoning, so the byte denominator is DRAM-visible traffic rather than all source-level loads and stores; cache write-back behavior, replay effects, and profiler noise can therefore influence the target quantity, which we mitigate but do not eliminate by averaging three first-invocation profiles. Closed-model behavior can also drift across snapshots, and a single query per input cannot establish reliability. We address the latter directly: rather than disclaim run-to-run variability, RQ3 measures it on a stratified repeated-query panel and reports where single-shot use is and is not safe.

Baseline limitations. The static baselines are deliberately modest. The deterministic references use approximate lexical and instruction-mix counts rather than compiler-grade analyzers, and the learned tree baselines are evaluated with grouped cross-validation on this corpus rather than a separately curated external split (though they remain stable across five seeds). Stronger compiler-driven or larger-scale learned baselines could therefore outperform the references reported here.

Failure-analysis validity and dataset leakage. The feature/error analysis is exploratory: its source-feature labels come from an auxiliary LLM-voting pipeline rather than human annotation, so we treat the feature/error association as descriptive evidence about recurring error patterns, not statistically validated causal evidence. Separately, because the models are off-the-shelf and trained on large web corpora, some kernels or their close variants may have appeared in pretraining data, and we cannot rule out such contamination completely. Both are reasons to frame this work as an empirical study rather than a claim about intrinsic reasoning ability in isolation.

VII. Conclusion

We framed LLM-based GPU performance reasoning as a software-engineering problem: deciding a kernel's BB/CB Roofline regime from software artifacts, before any profiling run, for developers and coding agents that cannot reach the target accelerator. Off-the-shelf LLMs already handle this well for the FP32 and FP64 regimes from source alone, but sit at chance for FP16 until cross-compiled SASS is supplied—after which FP16 classification becomes near-perfect for the stronger models. The decisive FP16 arithmetic is compiler-synthesized and invisible in source, so the source-versus-compilation boundary, not model choice, governs the hardest case; crucially, cross-compilation exposes it without any target-hardware access.

The signal is also practical to deploy. It is cheap and tiered—a budget model triages the FP32 regime for a fraction of a cent per query with no GPU—stable enough under repeated queries to act on, with the most accurate model also the most consistent, and it beats lightweight static baselines in the high-value regimes. No single configuration wins everywhere, so the right deployment is a cost-tiered cascade that matches the model and evidence tier to the target precision, not an LLM-only oracle.

These models do not replace profilers, they do not predict runtime in general, and numeric RAI is reliable only in identifiable regimes; the cascade itself remains a routing hypothesis we have not yet validated end-to-end. What we show is narrower and useful: execution-free Roofline-regime classification is a well-posed software-engineering task, and off-the-shelf prompting already solves it in identifiable, cheaply-detectable evidence tiers.

References

[1] S. Williams, A. Waterman, and D. A. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Communications of The ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009, mAG ID: 2002555321.

[2] H. Li, H. Zhang, and A. E. Hassan, "The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering," Jul. 2025, arXiv:2507.15003 [cs].

[3] J. Gong, V. Voskanyan, P. Brookes, F. Wu, W. Jie, J. Xu, R. Giavrimis, M. Basios, L. Kanthan, and Z. Wang, "Language Models for Code Optimization: Survey, Challenges and Future Directions," Jan. 2025, arXiv:2501.01277 [cs].

[4] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini, "KernelBench: Can LLMs Write Efficient GPU Kernels?" Feb. 2025, arXiv:2502.10517 [cs].

[5] W. Chen, J. Zhu, Q. Fan, Y. Ma, and A. Zou, "CUDA-LLM: LLMs Can Write Efficient CUDA Kernels," Jun. 2025, arXiv:2506.09092 [cs].

[6] G. Bolet, G. Georgakoudis, H. Menon, K. Parasyris, N. Hasabnis, H. Estes, K. Cameron, and G. Oren, "Can Large Language Models Predict Parallel Code Performance?" in *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*. University of Notre Dame Conference Facilities Notre Dame IN USA: ACM, Jul. 2025, pp. 1–6.

[7] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2007.

[8] Z. Wang, C. Ramos, M. A. Awad, and K. Lowery, "Omniwise: Predicting GPU Kernels Performance with LLMs," Jun. 2025, arXiv:2506.20886 [cs].

[9] S. Lee, A. Phanishayee, and D. Mahajan, "Forecasting GPU Performance for Deep Learning Training and Inference," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, Mar. 2025, pp. 493–508.

[10] M.-K. Nguyen-Nhat, H. D. N. Do, H. T. Le, and T. T. Dao, "LLMPerf: GPU Performance Modeling meets Large Language Models," in *2024 32nd International Conference on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*. Piscataway, NJ, USA: IEEE, Oct. 2024, pp. 1–8.

[11] HeCBench Developers, "HeCBench/ at master · zjin-lcf/HeCBench," 2026.

[12] S. Hong and H. Kim, "An analytical model for a GPU architecture with memory-level and thread-level parallelism awareness," *SIGARCH Comput. Archit. News*, vol. 37, no. 3, pp. 152–163, Jun. 2009.

[13] S. S. Baghsorkhi, M. Delahaye, S. J. Patel, W. D. Gropp, and W.-m. W. Hwu, "An adaptive performance modeling tool for GPU architectures," in *Proceedings of the 15th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '10. New York, NY, USA: Association for Computing Machinery, Jan. 2010, pp. 105–114.

[14] R. Lim, B. Norris, and A. Malony, "Autotuning GPU Kernels via Static and Predictive Analysis," in *2017 46th International Conference on Parallel Processing (ICPP)*, Aug. 2017, pp. 523–532, iSSN: 2332-5690.

[15] G. Alavani, K. Varma, and S. Sarkar, "Predicting Execution Time of CUDA Kernel Using Static Analysis," in *2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCloud/SocialCom/SustainCom)*. Piscataway, NJ, USA: IEEE, Dec. 2018, pp. 948–955.

[16] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás, "GPU Static Modeling Using PTX and Deep Structured Learning," *IEEE Access*, vol. 7, pp. 159 150–159 161, 2019.

[17] A. Lopes, F. Pratas, L. Sousa, and A. Ilic, "Exploring GPU performance, power and energy-efficiency bounds with Cache-aware Roofline Modeling," in *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Apr. 2017, pp. 259–268.

[18] L. Braun, S. Nikas, C. Song, V. Heuveline, and H. Fröning, "A Simple Model for Portable and Fast Prediction of Execution Time and Power Consumption of GPU Kernels," *ACM Trans. Archit. Code Optim.*, vol. 18, no. 1, pp. 7:1–7:25, Dec. 2021.

[19] Y. Emonds, L. Braun, and H. Fröning, "CUDAsap: Statically-Determined Execution Statistics as Alternative to Execution-Based Profiling," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. Piscataway, NJ, USA: IEEE, May 2023, pp. 119–130.

[20] H.-Q. Tran, V.-S. Pham, T.-S. Pham, T.-D. Chu, A.-T. Mai, X. T. Nguyen, and T.-T. Dao, "Accurate Latency Predictor for Triton-Based GPU Kernels with PTX Features and XGBoost," in *2025 RIVF International Conference on Computing and Communication Technologies (RIVF)*. Piscataway, NJ, USA: IEEE, Dec. 2025, pp. 980–985, iSSN: 2473-0130.

[21] M. Amarís, R. Y. de Camargo, M. Dyab, A. Goldman, and D. Trystram, "A comparison of GPU execution time prediction using machine learning and analytical modeling," in *2016 IEEE 15th International Symposium on Network Computing and Applications (NCA)*, Oct. 2016, pp. 326–333.

[22] Q. Wang and X. Chu, "GPGPU Performance Estimation With Core and Memory Frequency Scaling," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 12, pp. 2865–2881, Dec. 2020.

[23] J. Lee, Y. Ha, S. Lee, J. Woo, J. Lee, H. Jang, and Y. Kim, "GCoM: a detailed GPU core model for accurate analytical modeling of modern GPUs," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA '22. New York, NY, USA: Association for Computing Machinery, Jun. 2022, pp. 424–436.

[24] A. Dey, A. Dhakal, T. Z. Islam, J.-S. Yeom, T. Patki, D. Nichols, A. Movsesyan, and A. Bhatele, "Relative Performance Prediction Using Few-Shot Learning," in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. Osaka, Japan: IEEE, Jul. 2024, pp. 1764–1769.

[25] K. P. Selvam, P. M. Phothilimthana, S. Abu-El-Haija, B. Perozzi, and M. Brorsson, "Can LLMs Enhance Performance Prediction for Deep Learning Models?" Oct. 2024.

[26] K. Panner Selvam, "Performance Prediction Models for Deep Learning: A Graph Neural Network and Large Language Model Approach," 2025.

[27] N. R. Shah, M. V. V. S. M. Kumar, D. Baxi, and R. Upadrasta, "PolyUFC: Polyhedral Compilation Meets Roofline Analysis for Uncore Frequency Capping," in *Proceedings of the 2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2026.

[28] S. Rajput, A. Brandt, V. Elisseev, and T. Sharma, "FlipFlop: A Static Analysis-based Energy Optimization Framework for GPU Kernels," in *Proceedings of the 48th International Conference on Software Engineering (ICSE)*, 2026.

[29] Tal Kadosh, Niranjan Hasabnis, Vy Vo, Nadav Schneider, Neva Krien, Abdul Wasay, Nesreen K. Ahmed, Ted Willke, Gil Tamir, Yuval Pinter, Timothy G. Mattson, and Gal Oren, "Scope is all you need: Transforming LLMs for HPC Code," *arXiv.org*, Aug. 2023, aRXIV_ID: 2308.09440 MAG ID: 4386043885 S2ID: d5bf55f810cf66d3e70155bca12420d38d0601ad.

[30] Tal Kadosh, N. Hasabnis, Vy A. Vo, Nadav Schneider, Neva Krien, Mihai Capota, Abdul Wasay, Nesreen K. Ahmed, Ted Willke, G. Tamir, Yuval Pinter, Timothy Mattson, and Gal Oren, "MonoCoder: Domain-Specific Code Language Model for HPC Codes and Tasks," *IEEE Conference on High Performance Extreme Computing*, 2023, aRXIV_ID: 2312.13322 S2ID: f8c50d9bc22dea85bddf1859df7b11133a4f1c79.

[31] D. Nichols, A. Marathe, H. Menon, T. Gamblin, and A. Bhatele, "HPC-Coder: Modeling Parallel Programs using Large Language Models," in *ISC High Performance 2024 Research Paper Proceedings (39th International Conference)*, May 2024, pp. 1–12.

[32] Daniel Nichols, Aniruddha Marathe, Harshitha Menon, T. Gamblin, and A. Bhatele, "Modeling Parallel Programs using Large Language Models," *Information*

Security Conference, 2023, aRXIV_ID: 2306.17281 S2ID: 6ed21424a6ab0a5b0693adf36c607dbe78b5cf5d.

[33] C. Cummins, V. Seeker, D. Grubisic, B. Roziere, J. Gehring, G. Synnaeve, and H. Leather, "Meta Large Language Model Compiler: Foundation Models of Compiler Optimization," Jun. 2024, arXiv:2407.02524 [cs].

[34] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram, and A. Bhatele, "Can Large Language Models Write Parallel Code?" in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '24. New York, NY, USA: Association for Computing Machinery, Aug. 2024, pp. 281–294.

[35] Z. Wen, Y. Zhang, Z. Li, Z. Liu, L. Xie, and T. Zhang, "MultiKernelBench: A Multi-Platform Benchmark for Kernel Generation," Jul. 2025, arXiv:2507.17773 [cs].

[36] L. Chen, N. K. Ahmed, A. Dutta, A. Bhattacharjee, S. Yu, Q. I. Mahmud, W. Abebe, H. Phan, A. Sarkar, B. Butler, N. Hasabnis, G. Oren, V. A. Vo, J. P. Munoz, T. L. Willke, T. Mattson, and A. Jannesari, "The Landscape and Challenges of HPC Research and LLMs," Feb. 2024, arXiv:2402.02018 [cs].

[37] R. T. Lange, Q. Sun, A. Prasad, M. Faldor, Y. Tang, and D. Ha, "Towards Robust Agentic CUDA Kernel Benchmarking, Verification, and Optimization," Sep. 2025, arXiv:2509.14279 [cs].

[38] Bowen Cui, Tejas Ramesh, Oscar Hernandez, and Keren Zhou, "Do Large Language Models Understand Performance Optimization?" 2025.

[39] B. Cui, T. Ramesh, O. Hernandez, and K. Zhou, "Comprehensive Evaluation of LLMs in HPC Code Performance Optimization," in *Workshop Proceedings of the 54th International Conference on Parallel Processing*, ser. ICPP Workshops '25. New York, NY, USA: Association for Computing Machinery, Dec. 2025, pp. 1–8.

[40] Y. Peng, A. D. Gotmare, M. R. Lyu, C. Xiong, S. Savarese, and D. Sahoo, "PerfCodeGen: Improving Performance of LLM Generated Code with Execution Feedback," in *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. Piscataway, NJ, USA: IEEE, Apr. 2025, pp. 1–13.

[41] N. Purschke, S. Kirchner, and A. Knoll, "SpeedGen: Enhancing Code Efficiency through Large Language Model-Based Performance Optimization," in *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Mar. 2025, pp. 1–12.

[42] S. Damani, S. K. S. Hari, M. Stephenson, and C. Kozyrakis, "WarpDrive: An Agentic Workflow for Ninja GPU Transformations," *NeurIPS: Machine Learning For Systems Workshop*, 2024.

[43] M. Awad, C. Ramos, and K. Lowery, "IntelliPerf: LLM-Powered Autonomous GPU Performance Engineer," Jul. 2025.

[44] C. Hong, S. Bhatia, A. Cheung, and Y. S. Shao, "Autocomp: A Powerful and Portable Code Optimizer for Tensor Accelerators," Nov. 2025, arXiv:2505.18574 [cs].

[45] A. Tschand, M. Awad, R. Swann, K. Ramakrishnan, J. Ma, K. Lowery, G. Dasika, and V. J. Reddi, "SwizzlePerf: Hardware-Aware LLMs for GPU Kernel Performance Optimization," Aug. 2025, arXiv:2508.20258 [cs].

[46] Z. Zhang, R. Wang, S. Li, Y. Luo, M. Hong, and C. Ding, "CudaForge: An Agent Framework with Hardware Feedback for CUDA Kernel Optimization," Nov. 2025, arXiv:2511.01884 [cs].

[47] E. Kaplan, T. Bitan, L. Ghrayeb, L. Chen, T. Yotam, N. Hasabnis, and G. Oren, "ParaCodex: A Profiling-Guided Autonomous Coding Agent for Reliable Parallel Code Generation and Translation," Jan. 2026, arXiv:2601.04327 [cs].

[48] H. Peng, A. Gupte, R. Hasler, N. J. Eliopoulos, C.-C. Ho, R. Mantri, L. Deng, K. Läufer, G. K. Thiruvathukal, and J. C. Davis, "SysLLMatic: Large Language Models are Software System Optimizers," Oct. 2025, arXiv:2506.01249 [cs].

[49] R. Chu, A. Wang, X. Bai, S. Liu, and X. Dong, "GPU Kernel Optimization Beyond Full Builds: An LLM Framework with Minimal Executable Programs," Dec. 2025, arXiv:2512.22147 [cs].

[50] A. Wei, A. Nie, T. S. F. X. Teixeira, R. Yadav, W. Lee, K. Wang, and A. Aiken, "Improving Parallel Program Performance with LLM Optimizers via Agent-System Interfaces," 2024, version Number: 4.

[51] C. Baronio, P. Marsella, B. Pan, S. Guo, and S. Alberti, "Kevin: Multi-Turn RL for Generating CUDA Kernels," Jul. 2025, arXiv:2507.11948 [cs].

[52] X. Li, X. Sun, A. Wang, J. Li, and C. Shum, "CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning," Feb. 2026, arXiv:2507.14111 [cs].

[53] K. S. Dong, S. Modi, D. Nikiforov, S. Damani, E. Lin, S. K. S. Hari, and C. Kozyrakis, "KernelBlaster: Continual Cross-Task CUDA Optimization via Memory-Augmented In-Context Reinforcement Learning," Feb. 2026, arXiv:2602.14293 [cs].

[54] D. Nichols, K. Parasyris, C. Jekel, A. Bhatele, and H. Menon, "Integrating Performance Tools in Model Reasoning for GPU Kernel Optimization," Oct. 2025, arXiv:2510.17158 [cs].

[55] E. Hemberg, S. Moskal, and U.-M. O'Reilly, "Evolving Code with A Large Language Model," Jan. 2024, arXiv:2401.07102 [cs].

[56] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog, "AlphaEvolve: A coding agent for scientific and algorithmic discovery," Jun. 2025, arXiv:2506.13131 [cs].

[57] IEEE Computer Society, "IEEE Standard for Floating-Point Arithmetic," *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, Jul. 2019.

[58] N. Louvet, J.-M. Muller, and A. Panhaleux, "Newton-Raphson algorithms for floating-point division using an FMA," in *ASAP 2010 - 21st IEEE International Conference on Application-specific Systems, Architectures and Processors*, Jul. 2010, pp. 200–207.

[59] C. Liu, S. Lu, W. Chen, D. Jiang, A. Svyatkovskiy, S. Fu, N. Sundaresan, and N. Duan, "Code Execution with Pre-trained Language Models," May 2023, arXiv:2305.05383 [cs].

[60] C. Lyu, L. Yan, R. Xing, W. Li, Y. Samih, T. Ji, and L. Wang, "Large Language Models as Code Executors: An Exploratory Study," Oct. 2024, arXiv:2410.06667 [cs].

[61] Y. Li, H. Dhulipala, A. Yadavally, X. Rong, S. Wang, and T. N. Nguyen, "Blended Analysis for Predictive Execution," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 2987–3008, Jun. 2025.

[62] J. Chen, Z. Pan, X. Hu, Z. Li, G. Li, and X. Xia, "Reasoning Runtime Behavior of a Program with LLM: How Far are We?" in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, Apr. 2025, pp. 1869–1881.

[63] D. Xie, M. Zheng, X. Liu, J. Wang, C. Wang, L. Tan, and X. Zhang, "CORE: Benchmarking LLMs Code Reasoning Capabilities through Static Analysis Tasks," Jul. 2025, arXiv:2507.05269 [cs].

[64] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code," Jun. 2024, arXiv:2403.07974 [cs].

[65] S. Pyda, D. Nichols, and A. Bhatele, "The Shortcomings of Code LLMs in Modeling Code Properties," in *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*. Piscataway, NJ, USA: IEEE, May 2025, pp. 193–199.

[66] Z. Jin and J. S. Vetter, "A Benchmark Suite for Improving Performance Portability of the SYCL Programming Model," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Piscataway, NJ, USA: IEEE, Apr. 2023, pp. 325–327.

[67] T. Kadosh, N. Hasabnis, T. Mattson, Y. Pinter, and G. Oren, "Quantifying OpenMP: Statistical Insights into Usage and Adoption," in *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. Piscataway, NJ, USA: IEEE, Sep. 2023, pp. 1–7, iSSN: 2643-1971.